LESSON 06 • ASSIGNMENT

Homework 6
NumPy Fundamentals

AI/ML ENGINEERING BOOTCAMP / DATA ANALYSIS & PREPARATION

PART 01

Reminders (Xatırlatma)

Tapşırıqlara başlamazdan əvvəl dərsdə öyrəndiyimiz NumPy anlayışlarını və onların necə istifadə olunduğunu xatırlayaq.

1. NumPy Array

NumPy rəqəmsal məlumatlarla işləmək üçün istifadə olunan Python library-dir.

Python list-dən fərqli olaraq NumPy array-lər üzərində rəqəmsal hesablamaları daha rahat yerinə yetirə bilərik.

PYTHON

```python
import numpy as np

numbers = np.array([10, 20, 30, 40])

print(numbers)
print(type(numbers))
```

Output:

OUTPUT

```
[10 20 30 40]
<class 'numpy.ndarray'>
```

Python List vs NumPy Array:

PYTHON

```python
numbers_list = [10, 20, 30]
numbers_array = np.array([10, 20, 30])

print(numbers_list * 2)
print(numbers_array * 2)
```

Output:

OUTPUT

```
[10, 20, 30, 10, 20, 30]
[20 40 60]
```

Python list-də `* 2` elementləri təkrarlayır. NumPy array-də isə hər element 2-yə vurulur.

2. Array Properties

NumPy array-in quruluşunu öyrənmək üçün bir neçə attribute istifadə edirik.

PYTHON

```python
data = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

print(data.shape)
print(data.ndim)
print(data.size)
print(data.dtype)
```

Output:

OUTPUT

```
(2, 3)
2
6
int64
```

`dtype` sistemdən asılı olaraq `int32` də ola bilər.

- `shape` - Array-in hər dimension üzrə ölçüsü. Burada 2 row və 3 column var.
- `ndim` - Dimension sayı. Burada 2D array-dir.
- `size` - Ümumi element sayı. Burada 6-dır.
- `dtype` - Elementlərin data type-ı.

3. Indexing və Slicing

Indexing konkret elementə, slicing isə array-in müəyyən hissəsinə müraciət etmək üçündür.

PYTHON

```python
data = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])

print(data[0, 1])
print(data[2])
print(data[:, 0])
print(data[0:2, 1:3])
```

Output:

```
OUTPUT
20

[70 80 90]

[10 40 70]

[[20 30]
 [50 60]]
```

Yadda saxlayın:

- `data[0, 1]` - Birinci row, ikinci column.
- `data[2]` - Üçüncü row.
- `data[:, 0]` - Bütün row-ların birinci column-u.
- `data[0:2, 1:3]` - İlk iki row-un ikinci və üçüncü column-ları.
- `data[-1, -1]` - Sonuncu row-un sonuncu elementi.

Index-lər 0-dan başlayır. Slicing-də stop index daxil edilmir.

4. Vectorized Operations

NumPy array üzərində arithmetic operations tətbiq etdikdə əməliyyat hər elementə tətbiq olunur.

```python
PYTHON
numbers = np.array([10, 20, 30])

print(numbers + 5)
print(numbers * 2)
print(numbers / 2)
```

Output:

```
OUTPUT
[15 25 35]
[20 40 60]
[ 5. 10. 15.]
```

İki array arasında da element-wise əməliyyat aparmaq mümkündür.

```python
PYTHON
first = np.array([10, 20, 30])
second = np.array([15, 25, 40])

print(second - first)
```

Output:

OUTPUT

```
[ 5  5 10]
```

Burada eyni mövqedə yerləşən elementlər bir-birindən çıxılır.

5. Boolean Indexing və Filtering

Comparison operators vasitəsilə array-in elementlərini müəyyən şərtə əsasən yoxlaya və seçə bilərik.

PYTHON

```python
numbers = np.array([45, 60, 75, 90, 55])

mask = numbers >= 60

print(mask)
print(numbers[mask])
```

Output:

OUTPUT

```
[False  True  True  True False]
[60 75 90]
```

Burada `mask` hansı elementlərin şərti ödədiyini göstərir.

Şərti ödəyən elementlərin sayını da tapa bilərik:

PYTHON

```python
print((numbers >= 60).sum())
```

Output:

OUTPUT

```
3
```

Çünki `True` hesablamalarda 1, `False` isə 0 kimi qəbul olunur.

6. Statistical Operations

NumPy-də array üzərində statistik hesablamalar aparmaq üçün hazır operations var.

PYTHON

```python
numbers = np.array([10, 20, 30, 40, 100])

print(numbers.sum())
print(numbers.mean())
print(numbers.min())
print(numbers.max())
print(np.median(numbers))
```

Output:

```
OUTPUT
200
40.0
10
100
30.0
```

- `sum()` - Bütün elementlərin cəmi.
- `mean()` - Arithmetic mean (ortalama).
- `min()` - Minimum dəyər.
- `max()` - Maksimum dəyər.
- `np.median()` - Median dəyər.

Mean və Median fərqi:

Bu nümunədə mean `40`, median isə `30`-dur. Çünki `100` kimi böyük dəyər mean nəticəsini artırır. Median isə sıralanmış dəyərlərin orta mövqeyinə əsaslanır.

7. Axis - Row və Column üzrə hesablamalar

2D array-lərdə `axis` hesablamanın hansı istiqamətdə aparılacağını müəyyən edir.

```python
data = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

print(data.mean(axis=0))
print(data.mean(axis=1))
```

Output:

```
OUTPUT
[25. 35. 45.]
[20. 50.]
```

`axis=0` - Hər column üzrə bir nəticə hesablayır.

`axis=1` - Hər row üzrə bir nəticə hesablayır.

Eyni məntiq `sum()`, `min()`, `max()` kimi operations üçün də keçərlidir.

8. Copy və Original Array

NumPy-də array üzərində hesablamalar apararkən orijinal məlumatın dəyişib-dəyişmədiyini nəzərə almaq vacibdir.

PYTHON

```python
numbers = np.array([10, 20, 30])

updated = numbers + 5

print(numbers)
print(updated)
```

Output:

OUTPUT

```
[10 20 30]
[15 25 35]
```

Burada yeni array yaradılır və orijinal array dəyişmir.

Amma slicing nəticəsində alınan view üzərində dəyişiklik etdikdə əsas array də dəyişə bilər.

Müstəqil copy yaratmaq üçün:

PYTHON

```python
numbers = np.array([10, 20, 30])

new_numbers = numbers[0:2].copy()

new_numbers[0] = 100

print(numbers)
print(new_numbers)
```

Output:

OUTPUT

```
[10 20 30]
[100  20]
```

PART 02

Tasks

Praktik tapşırıqlar - eyni dataset üzərində addım-addım NumPy əməliyyatları.

Bütün tapşırıqlar üçün aşağıdakı dataset-dən istifadə edin.

Dataset 5 tələbənin 3 imtahan üzrə nəticələrini göstərir.

PYTHON

```python
import numpy as np

names = ["Aysel", "Murad", "Nigar", "Orxan", "Leyla"]

scores = np.array([
    [72, 80, 88],
    [60, 74, 68],
    [95, 91, 89],
    [45, 55, 65],
    [82, 78, 84]
])
```

> **Qeyd:** Hər row bir tələbəyə, hər column isə bir imtahana uyğundur. Tapşırıqlar boyunca `scores` array-indəki orijinal balları dəyişməyin, yalnız tapşırıq ayrıca bunu tələb edirsə, copy üzərində işləyin.

Task 1 - Array Properties

1. `scores` array-inin `shape` dəyərini göstərin.
2. Array-in neçə dimension-dan ibarət olduğunu tapın.
3. Ümumi element sayını göstərin.
4. Array-in `dtype` dəyərini göstərin.
5. `shape` attribute-undan istifadə edərək tələbə sayını və imtahan sayını ayrıca əldə edin.

Output-da nəticələri aydın göstərməyə çalışın:

OUTPUT

```
Number of students: ...
Number of exams: ...
Total values: ...
Dimensions: ...
Data type: ...
```

> **Hint:** `shape` nəticəsindəki dəyərlərə indexing ilə müraciət edə bilərsiniz.

Task 2 - Indexing

Indexing istifadə edərək aşağıdakı məlumatları əldə edin:

1. Ayselin birinci imtahan balı.
2. Muradın üçüncü imtahan balı.
3. Nigarın ikinci imtahan balı.
4. Leylanın sonuncu imtahan balı.
5. Orxanın bütün imtahan nəticələri.
6. Sonuncu tələbənin sonuncu imtahan balını negative indexing ilə əldə edin.

Hər nəticəni uyğun tələbə və imtahan adı ilə göstərin.

> **Hint:** 2D array üçün `scores[row, column]` sintaksisindən istifadə edin.

Task 3 - Slicing

Slicing istifadə edərək aşağıdakı məlumatları seçin:

1. İlk 3 tələbənin bütün imtahan nəticələri.
2. Son 2 tələbənin bütün imtahan nəticələri.
3. Bütün tələbələrin birinci imtahan nəticələri.
4. Bütün tələbələrin üçüncü imtahan nəticələri.
5. İlk 3 tələbənin yalnız ikinci və üçüncü imtahan nəticələri.
6. Murad və Nigarın bütün imtahan nəticələri.

Əlavə olaraq, 5-ci bənddə əldə etdiyiniz array-in `shape` dəyərini göstərin.

Task 4 - Vectorized Operations

NumPy vectorized operations istifadə edərək aşağıdakı əməliyyatları yerinə yetirin:

1. Bütün imtahan ballarına 5 əlavə edin və nəticəni `updated_scores` adlı yeni array-də saxlayın.
2. Bütün ballardan 10 çıxın.
3. Bütün balları 2-yə bölün.
4. Bütün balların 2-yə vurulmuş variantını yaradın.
5. Birinci və ikinci imtahan nəticələrinin fərqini hesablayın (`Exam 2 - Exam 1`).
6. İkinci və üçüncü imtahan nəticələrinin fərqini hesablayın (`Exam 3 - Exam 2`).

Sonda orijinal `scores` array-ini ekrana çıxarıb dəyişmədiyini yoxlayın.

Task 5 - Boolean Indexing

Boolean indexing və comparison operators vasitəsilə:

1. 70-dən aşağı olan bütün balları seçin.
2. 70 və daha yüksək olan bütün balları seçin.
3. 90 və daha yüksək olan balları seçin.
4. 50-dən aşağı olan balları seçin.

5. 80-dən yüksək olan balları seçin.

Daha sonra hər şərt üçün neçə elementin uyğun gəldiyini hesablayın.

Expected Output formatı:

```
OUTPUT
Scores below 70: [...]
Count below 70: ...

Scores 90 or above: [...]
Count 90 or above: ...
```

> **Hint:** Əvvəl boolean mask yaradıb həmin mask üzərində `sum()` istifadə edə bilərsiniz.

Task 6 - Statistical Operations

`scores` dataset-i üzərində aşağıdakı hesablamaları aparın:

1. Bütün balların cəmini hesablayın.
2. Bütün dataset-in ümumi ortalamasını tapın.
3. Ən yüksək balı tapın.
4. Ən aşağı balı tapın.
5. Bütün balların median dəyərini hesablayın.
6. Ən yüksək və ən aşağı bal arasındakı fərqi hesablayın.
7. Ümumi ortalamadan yüksək olan balları seçin və onların sayını göstərin.

> **Hint:** `mean()` ilə əldə etdiyiniz nəticəni comparison zamanı istifadə edə bilərsiniz.

Task 7 - Axis Operations

Bu tapşırıqda `axis=0` və `axis=1` istifadə etməlisiniz.

1. Hər tələbənin ortalama balını hesablayın.
2. Hər imtahanın ortalama balını hesablayın.
3. Hər tələbənin ən yüksək imtahan balını tapın.
4. Hər tələbənin ən aşağı imtahan balını tapın.
5. Hər imtahan üzrə ən yüksək balı tapın.
6. Hər imtahan üzrə ən aşağı balı tapın.
7. Hər tələbənin 3 imtahan üzrə toplam balını hesablayın.

Expected Output formatı:

OUTPUT

```
Student averages: [...]
Exam averages: [...]

Highest score per student: [...]
Lowest score per student: [...]

Highest score per exam: [...]
Lowest score per exam: [...]

Total score per student: [...]
```

> **Diqqət:** Hansı hesablamada `axis=0`, hansında `axis=1` istifadə etməli olduğunuzu özünüz müəyyənləşdirin.

Task 8 - Student Filtering

Bu tapşırıqda tələbələri ortalama nəticələrinə görə filter edəcəksiniz.

1. Hər tələbənin ortalama balını hesablayın.
2. Ortalaması 75 və ya daha yüksək olan tələbələri müəyyən edin.
3. Bu şərti ödəyən tələbələrin sayını hesablayın.
4. Ortalaması 75-dən aşağı olan tələbələrin sayını tapın.
5. Ortalaması 85-dən yüksək olan tələbələrin bütün imtahan nəticələrini seçin.
6. Ən yüksək ortalamaya sahib tələbənin ortalama balını tapın.

> **Hint:** Əvvəl student averages array-i yaradın. Sonra həmin array üzərində boolean indexing tətbiq edin.